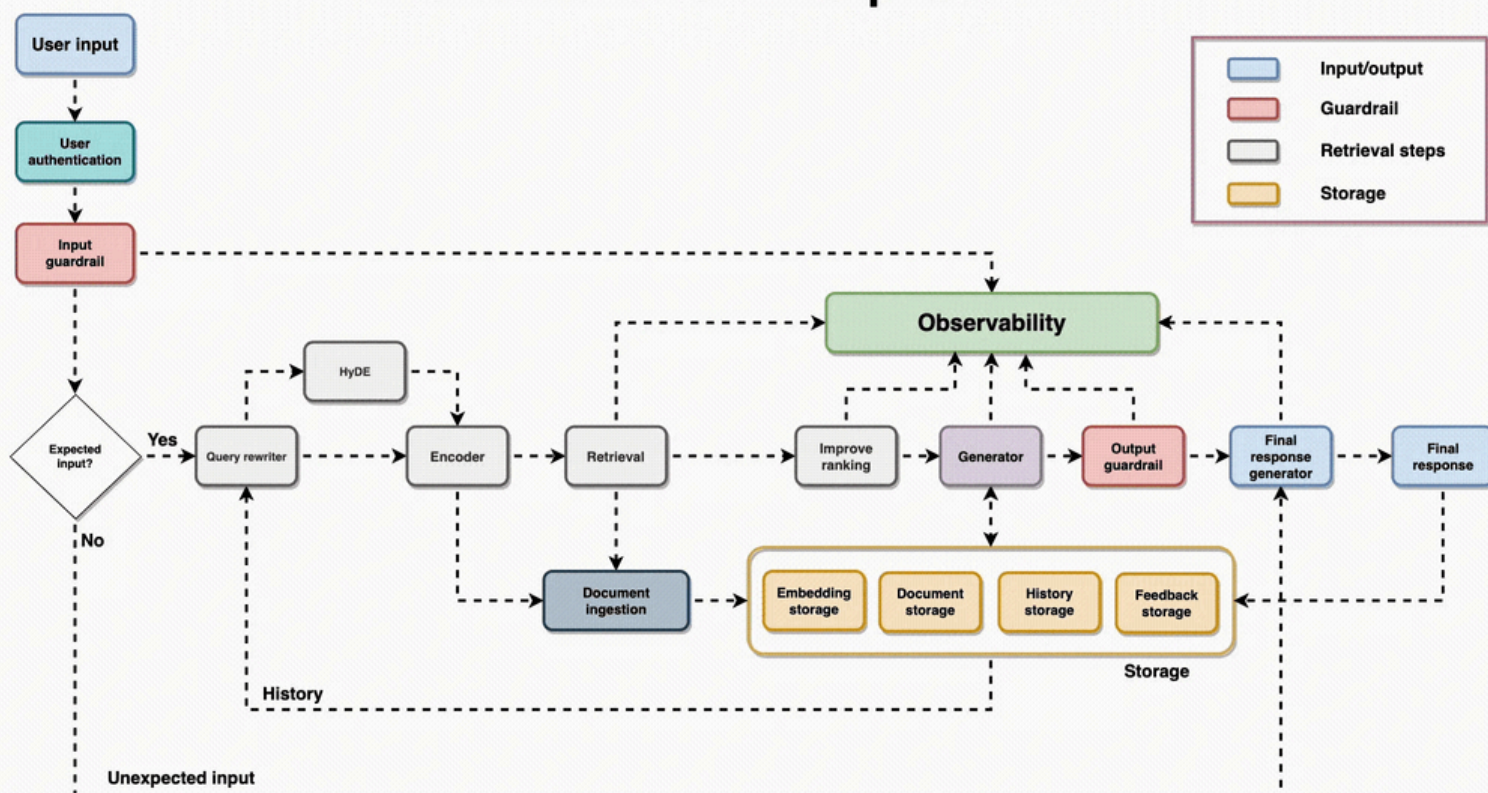


# RAG

## Retrieval-Augmented Generation

## Complete Cheatsheet →

### Architecture For Enterprise RAG



# What is RAG?

**Definition:** RAG combines retrieval systems with AI language models to provide accurate, contextual answers by fetching relevant information from external knowledge sources.

## How It Works:

User Query → Retrieve Relevant Docs → Combine with Query → LLM → Answer

## Why RAG?

- ✓ Reduces AI hallucinations
- ✓ Always up-to-date information
- ✓ Works with your private data
- ✓ No need to retrain models
- ✓ Cost-effective solution

## Common Use Cases:

- Customer support chatbots
- Internal knowledge bases
- Document Q&A systems
- Research assistants
- Code documentation helpers

# RAG Architecture - The Big Picture

---

## Three Main Stages:

### 1. Indexing (Offline - Done Once)

- Load documents (PDF, DOCX, HTML, etc.)
- Split into smaller chunks
- Convert chunks to embeddings (vectors)
- Store in vector database

### 2. Retrieval (Real-time)

- Convert user query to embedding
- Search vector DB for similar chunks
- Rank and select top-K results

### 3. Generation (Real-time)

- Combine query + retrieved context
- Send to LLM
- Generate final answer

**Key Point:** The magic is in connecting these three stages seamlessly!

# Document Processing & Chunking

---

## Step 1: Load Documents

- Supported formats: PDF, DOCX, TXT, Markdown, HTML, CSV
- Tools: PyPDF2, python-docx, LangChain loaders

## Step 2: Chunking Strategies

### Fixed-Size Chunking

Split by character/token count (e.g., 500 tokens). Add overlap (50-100 tokens). Simple but may break context.

### Semantic Chunking

Split by paragraphs or sentences. Preserves meaning better. Variable sizes.

## Best Practices:

- Chunk size: 300-800 tokens (sweet spot: 500)
- Overlap: 10-20% of chunk size
- Keep metadata: source file, page number, section
- Test different sizes for your use case

# Embeddings Explained

**What Are Embeddings?** Vector representations of text that capture meaning. Similar texts have similar vectors.

## Example:

```
"cat" → [0.2, 0.8, 0.1, ...]  
"kitten" → [0.3, 0.7, 0.2, ...] (very close!)  
"car" → [0.9, 0.1, 0.8, ...] (far away)
```

## Popular Models:

### Open-Source:

→ **all-MiniLM-L6-v2** → 384 dimensions, fast, general use

→ **bge-large-en-v1.5** → 1024 dimensions, high quality

### Commercial:

→ OpenAI **text-embedding-3-small** → 1536 dimensions

→ Cohere **embed-english-v3.0** → 1024 dimensions

**Choosing a Model:** Higher dimensions = better accuracy but slower. Match domain (medical, legal, general). Consider speed vs quality tradeoff.

# Vector Databases

**What They Do:** Store and search millions of high-dimensional vectors efficiently using similarity metrics.

## Popular Options:

### Cloud/Managed:

- **Pinecone** - Easy, fully managed
- **Weaviate** - Open-source + cloud
- **Qdrant** - Fast, modern

### Self-Hosted:

- **ChromaDB** - Lightweight, great for dev
- **FAISS** - Facebook's library, very fast
- **Milvus** - Enterprise-scale

## Similarity Metrics:

- **Cosine Similarity** → Measures angle between vectors (best for text)
- **Euclidean Distance** → Straight-line distance
- **Dot Product** → For normalized vectors

# Retrieval Strategies

6

## Basic Retrieval:

Query → Embedding → Vector Search → Top-K Documents

## Advanced Techniques:

### 1. Hybrid Search

Combines semantic search + keyword (BM25) for better coverage

### 2. Re-ranking

→ First pass: Retrieve 100 documents (fast)

→ Second pass: Re-rank to top 10 (precise)

→ Uses specialized models

### 3. Metadata Filtering

Filter by date, author, category before searching

### 4. Maximal Marginal Relevance (MMR)

Balances relevance with diversity to avoid redundant results

## Key Parameters:

→ **Top-K:** Usually 3-10 documents

→ **Score Threshold:** Minimum similarity ((0.7-0.8))

→ **Fetch-K:** Initial retrieval count for re-ranking

# LLM Integration & Prompting

7

## Popular LLMs for RAG:

**Commercial:** GPT-4, Claude 3, Gemini Pro

**Open-Source:** Llama 3, Mistral, Mixtral

## RAG Prompt Template:

Context Information:

{retrieved\_documents}

User Question: {query}

Instructions: Answer based ONLY on the context above.

If not found, say "I don't have enough information."

Answer:

## Prompt Engineering Tips:

- **Be explicit:** "Use only provided context"
- **Request citations:** "Cite source documents"
- **Set temperature low (0-0.3)** for factual answers
- **Add examples** for better results
- **Handle uncertainty gracefully**

**Context Window:** Match retrieval amount to model limits (GPT-4: 128K, Claude: 200K tokens)

# RAG Frameworks

8

## **LangChain** (Most Popular)

- Comprehensive ecosystem
- 300+ integrations
- Chains, agents, memory
- Great for complex workflows

## **LlamaIndex** (Data-Focused)

- Index management
- Query engines
- Best for structured data
- Excellent documentation

## **Haystack** (Production-Ready)

- Pipeline-based
- Easy deployment
- Question answering focus

## Quick Setup Example (LangChain):

```
from langchain.chains import RetrievalQA
from langchain.vectorstores import Chroma

qa = RetrievalQA.from_chain_type(
    llm=your_llm,
    retriever=vectorstore.as_retriever()
)
result = qa("Your question?")
```

**Choose Based On:** LangChain → Complex apps. LlamaIndex → Data-centric. Haystack → Production QA systems

# Evaluation Metrics

*Why Evaluate?* Measure and improve your RAG system's performance.

## Retrieval Metrics:

- **Precision@K:** Relevant docs in top-K results
- **Recall@K:** Coverage of all relevant docs
- **MRR:** Rank of first relevant result

## Generation Metrics:

- **Faithfulness:** Answer matches context (no hallucinations)
- **Answer Relevance:** Addresses the question
- **Context Relevance:** Retrieved docs are useful

## Popular Tools:

- **RAGAS:** Complete RAG evaluation framework
- **TruLens:** LLM app monitoring
- **DeepEval:** Testing suite

## Example:

```
from ragas import evaluate
result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy]
)
```

**Golden Rule:** Test on real user queries regularly!

# Advanced RAG Techniques

## 1. Multi-Query RAG

Generate multiple query variations → Retrieve for each → Combine results. **Better recall for ambiguous queries**

## 2. Self-RAG

LLM reflects on retrieved quality → Decides to use, ignore, or re-retrieve. **Self-correcting system**

## 3. Agentic RAG

LLM autonomously decides when/what to retrieve with multi-step reasoning. **Most flexible approach**

## 4. Parent-Child Chunking

Search small chunks → Retrieve larger parent context. **Balance precision + context**

## 5. GraphRAG

Combine knowledge graphs + vector search. **Better for complex, relationship-heavy queries**

**When to Use Advanced:** Complex domains (legal, medical), multi-hop reasoning needed, high accuracy requirements.

# Common Challenges & Solutions

11

## Challenge 1: Poor Retrieval

*Wrong chunks returned*

*Fix: Better chunking, hybrid search, tune similarity threshold*

## Challenge 2: Hallucinations

*LLM invents information*

*Fix: Stricter prompts, lower temperature, require citations*

## Challenge 3: Slow Responses

*Takes too long to answer*

*Fix: Cache queries, faster models, reduce top-K, optimize indexing*

## Challenge 4: Context Overflow

*Too much text for LLM*

*Fix: Better re-ranking, summarize chunks, use larger context models*

## Challenge 5: Outdated Info

*Results not current*

*Fix: Regular re-indexing, incremental updates, timestamp filters*

**Pro Tip:** Monitor these metrics continuously in production!

# Quick Implementation Guide

---

## 5-Step Setup:

### Step 1: Install

```
pip install langchain openai chromadb tiktoken
```

### Step 2: Load & Chunk

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

docs = PyPDFLoader("file.pdf").load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(docs)
```

### Step 3: Create Vector Store

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

vectordb = Chroma.from_documents(
    chunks, OpenAIEmbeddings()
)
```

### Step 4: Build RAG Chain

```
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vectordb.as_retriever()
)
```

### Step 5: Query

```
result = qa("Your question?")
print(result)
```

# Production Best Practices

---

## Architecture:

- Separate indexing (offline) from querying (online)
- Use async processing for large document loads
- Implement caching (Redis) for common queries
- Load balance LLM API calls

## Security:

- Encrypt vectors at rest and in transit
- Implement access controls (RBAC)
- Sanitize user inputs
- Handle PII carefully (anonymize or redact)

## Monitoring:

Track these metrics:

- Query latency (p95, p99)
- Retrieval accuracy
- LLM response quality
- Error rates
- Cost per query

## Cost Optimization:

- Cache embeddings and responses

## Customer Support

- Answer FAQs from documentation
- Multi-turn conversations
- Ticket resolution

## Enterprise Knowledge

- Internal wiki search
- Policy/procedure lookup
- Onboarding assistants

## Legal & Compliance

- Contract analysis
- Regulatory research
- Case law search

## Healthcare

- Medical literature Q&A
- Clinical decision support
- Drug interaction checks

## Education

- Personalized tutoring
- Research assistance
- Study guides

# Learning Path & Resources

15

## Your 4-Week Journey:

### Week 1: Fundamentals

- Understand embeddings and vector search
- Try LangChain tutorials
- Build basic RAG demo

### Week 2: Intermediate

- Experiment with chunking strategies
- Implement hybrid search
- Add evaluation metrics

### Week 3: Advanced

- Multi-query RAG
- Re-ranking techniques
- Prompt optimization

### Week 4: Production

- Deployment strategies
- Monitoring setup
- Performance tuning

## Essential Resources:

- **Docs:** LangChain, LlamaIndex official docs
- **Courses:** DeepLearning.AI LangChain course